# Experiment 1 ? Program 1

# Data Science - 22DS102006

# 1. Working with Different Data Formats using pandas

a. Reading and Writing CSV Data

```
import pandas as pd

# Read CSV file from URL
url = 'https://people.sc.fsu.edu/~jburkardt/data/csv/hw_200.csv'
df = pd.read_csv(url)
print(df.head())



# Write CSV to file
df.to_csv("output.csv", index=False)
```

b. Reading and Writing JSON Data

```
# Convert DataFrame to JSON and back
json_data = df.to_json()
df_json = pd.read_json(json_data)
print(df_json.head())
```

c. Reading and Writing Excel Files

```
# Save and load Excel files
df.to_excel("output.xlsx", index=False)
df_excel = pd.read_excel("output.xlsx")
print(df_excel.head())
```

# Experiment:

# Working with Different Data Formats Using Pandas

# Perform reading and writing data in text format using read_csv and read_table considering any online dataset in delimited format (CSV).

**Perform reading, writing and parsing data in JSON (Javascript Object Notation) format using read_json.**

**Perform reading and writing of Microsoft Excel Files (xslx) using read_excel.**

## Aim

# Working with Different Data Formats Using Pandas

# Perform reading and writing data in text format using read_csv and read_table considering any online dataset in delimited format (CSV).

To perform reading, writing, and parsing operations on CSV, tab-delimited text, JSON, and Microsoft Excel files using the Pandas library in Python.

To perform reading and writing operations on text files using Pandas functions read_csv() and read_table() by considering an online CSV dataset.

## Software Requirements

Python 3

Pandas library

OpenPyXL library for Excel files

Jupyter Notebook, Google Colab, or any Python IDE

Install the required libraries using:

```
pip install pandas openpyxl
```

# Syntax

### Reading a CSV File

```
pd.read_csv(filepath)
```

### Reading a Tab-Delimited File

```
pd.read_table(filepath)
```

### Writing Data into a CSV File

```
DataFrame.to_csv(filename, index=False)
```

# A. Reading and Writing Data in Text Format

Pandas provides the read_csv() function to read CSV and other delimited text files.

## Case 1: Reading an Online CSV Dataset

```
import pandas as pd

# Online CSV dataset

url = "https://raw.githubusercontent.com/mwaskom/seaborn-data/master/iris.csv"

# Reading the CSV file

df = pd.read_csv(url)

# Displaying the first five rows

print("First five rows of the dataset:")

print(df.head())

# Displaying dataset information

print("\nDataset Information:")

print(df.info())

# Displaying the shape of the dataset

print("\nNumber of rows and columns:")

print(df.shape)
```

## Sample Output

```
First five rows of the dataset:

   sepal_length  sepal_width  petal_length  petal_width species

0           5.1          3.5           1.4          0.2  setosa
```

```
1          4.9          3.0          1.4          0.2  setosa
2          4.7          3.2          1.3          0.2  setosa
3          4.6          3.1          1.5          0.2  setosa
4          5.0          3.6          1.4          0.2  setosa
Number of rows and columns:
(150, 5)
```

## Case 2: Writing Data into a CSV File

```
import pandas as pd
# Reading the online CSV dataset
url = "https://raw.githubusercontent.com/mwaskom/seaborn-data/master/iris.csv"
df = pd.read_csv(url)
# Writing the dataset into a new CSV file
df.to_csv("iris_output.csv", index=False)
print("The dataset has been successfully written to iris_output.csv")
```

# Output

```
CSV file created successfully.
```

A new file named iris_output.csv is created in the current working directory.

### Explanation

to_csv() writes a DataFrame into a CSV file.

index=False prevents Pandas from writing row index values into the file.

## Case 3: Reading a Tab-Delimited File using read_table()

Create a file named students.txt with the following tab-separated data:

```
Roll_No Name    Department      Percentage
101     Anusha          IT      89
102     Babitha         IT      92
```

```
103     Charitha        CSE     88

104     Deepika         ECE     85

(Values are separated by TAB spaces.)
```

Python program:

```
import pandas as pd

# Reading a tab-delimited text file

df = pd.read_csv("students.txt", sep="\t")

print("Student data:")

print(df)
```

Alternatively, read_table() can be used:

```
import pandas as pd

# read_table uses tab as the default delimiter

df = pd.read_table("students.txt")

print(df)
```

## Sample Output

```
   Roll_No      Name Department  Percentage

0      101    Anusha         IT          89

1      102   Babitha         IT          92

2      103  Charitha        CSE          88

3      104   Deepika        ECE          85
```

## Case 4: Writing Data into a Tab-Delimited File

```
import pandas as pd

student_data = {

    "Roll_No": [101, 102, 103, 104],

    "Name": ["Anusha", "Babitha", "Charitha", "Deepika"],

    "Department": ["IT", "IT", "CSE", "ECE"],
```

```
    "Percentage": [89, 92, 88, 85]
}
df = pd.DataFrame(student_data)
# Writing data into a tab-delimited file
df.to_csv("student_output.txt", sep="\t", index=False)
print("Data successfully written to student_output.txt")
```

# Output

```
Tab-delimited text file created successfully.
```

A new file named students_output.txt is created.

# B. Reading, Writing, and Parsing JSON Data

**Perform reading, writing and parsing data in JSON (Javascript Object Notation) format using read_json.**

JSON stands for JavaScript Object Notation. It stores data in key-value form.

# Aim

To perform reading, writing, and parsing operations on JSON (JavaScript Object Notation) data using the Pandas library.

# Software Requirements

Python 3.x

Pandas Library

Jupyter Notebook / Google Colab / VS Code

# Theory

JSON (JavaScript Object Notation) is a lightweight data-interchange format used to store and exchange structured data. It stores data as key-value pairs and is commonly used in web applications and APIs.

Pandas provides:

read_json() – Reads JSON data into a DataFrame.

to_json() – Writes a DataFrame into a JSON file.

# Syntax

### Reading a JSON File

```
pd.read_json("filename.json")
```

### Writing Data into a JSON File

```
DataFrame.to_json("filename.json", orient="records", indent=4)
```

# Case 1: Writing Data into a JSON File

```
import pandas as pd

# Create a dictionary

student_data = {

    "Roll_No": [101, 102, 103, 104],

    "Name": ["Anusha", "Babitha", "Charitha", "Deepika"],

    "Department": ["IT", "IT", "CSE", "ECE"],

    "Marks": [89, 92, 88, 85]

}

# Convert dictionary to DataFrame

df = pd.DataFrame(student_data)

# Write DataFrame to JSON file

df.to_json(

    "students.json",

    orient="records",

    indent=4

)
```

```
print("JSON file created successfully.")
```

# Output

```
JSON file created successfully.
```

The generated students.json file will look like:

```
[
    {
        "Roll_No":101,
        "Name":"Anusha",
        "Department":"IT",
        "Marks":89
    },
    {
        "Roll_No":102,
        "Name":"Babitha",
        "Department":"IT",
        "Marks":92
    },
    {
        "Roll_No":103,
        "Name":"Charitha",
        "Department":"CSE",
        "Marks":88
    },
    {
        "Roll_No":104,
        "Name":"Deepika",
```

```
        "Department":"ECE",
        "Marks":85
    }
]
```

# Program 2: Reading a JSON File using read_json()

```
import pandas as pd
# Read JSON file
df = pd.read_json("students.json")
# Display the DataFrame
print("Student Data")
print(df)
```

# Sample Output

```
Student Data
   Roll_No      Name Department  Marks
0      101    Anusha         IT     89
1      102   Babitha         IT     92
2      103  Charitha        CSE     88
3      104   Deepika        ECE     85
```

# Program 3: Parsing JSON Data from a String

```
import pandas as pd
from io import StringIO
json_data = """
[
    {
```

```
        "Roll_No":101,
        "Name":"Anusha",
        "Marks":89
    },
    {
        "Roll_No":102,
        "Name":"Babitha",
        "Marks":92
    },
    {
        "Roll_No":103,
        "Name":"Charitha",
        "Marks":88
    }
]
"""
# Parse JSON string
df = pd.read_json(StringIO(json_data))
print("Parsed JSON Data")
print(df)
```

# Sample Output

```
Parsed JSON Data
   Roll_No      Name  Marks
0      101    Anusha     89
1      102   Babitha     92
```

```
2      103  Charitha     88
```

# Program 4: Reading JSON Data from an Online URL

```
import pandas as pd

url = "https://jsonplaceholder.typicode.com/users"

# Read JSON data from the URL

df = pd.read_json(url)

# Display first five records

print(df.head())
```

# Sample Output

```
   id           name         username              email

0   1   Leanne Graham          Bret    Sincere@april.biz

1   2   Ervin Howell      Antonette   Shanna@melissa.tv

2   3   Clementine Bauch Samantha     Nathan@yesenia.net

3   4   Patricia Lebsack Karianne    Julianne@...

4   5   Chelsey Dietrich Kamren      Lucio@...
```

# Explanation

pd.read_json() reads JSON data from a file, URL, or string and converts it into a DataFrame.

DataFrame.to_json() writes a DataFrame into a JSON file.

orient="records" stores each row as a separate JSON object.

indent=4 formats the JSON file with proper indentation for readability.

StringIO() allows JSON text stored in a string to be treated like a file for parsing.

# Applications

Reading API responses.

Exchanging data between web applications.

Storing structured data.

Data analysis using Pandas.

Importing and exporting JSON datasets.

# Result

Thus, reading, writing, and parsing of JSON data were successfully performed using the Pandas functions read_json() and to_json().

# C. Reading and Writing Microsoft Excel Files

# Aim

**Perform reading and writing of Microsoft Excel Files (xslx) using read_excel. Please write solutions for above lab.**

To perform reading and writing operations on Microsoft Excel (.xlsx) files using the Pandas library.

Pandas provides the read_excel() function to read Excel files and the to_excel() function to write Excel files.

# Software Requirements

Python 3.x

Pandas Library

OpenPyXL Library

Jupyter Notebook / Google Colab / VS Code

Installation

```
pip install pandas openpyxl
```

# Theory

Microsoft Excel is one of the most widely used spreadsheet applications for storing and managing data. Pandas provides functions to read data from Excel files and write DataFrames to Excel files.

The commonly used functions are:

read_excel() – Reads data from an Excel (.xlsx) file.

to_excel() – Writes a DataFrame into an Excel (.xlsx) file.

# Syntax

### Reading an Excel File

```
pd.read_excel("filename.xlsx")
```

### Writing to an Excel File

```
DataFrame.to_excel("filename.xlsx", index=False)
```

## Program 8: Creating and Writing an Excel File

```
import pandas as pd

student_data = {

    "Roll_No": [101, 102, 103, 104],

    "Name": ["Anusha", "Babitha", "Charitha", "Deepika"],

    "Department": ["IT", "IT", "CSE", "ECE"],

    "Percentage": [89, 92, 88, 85]

}

df = pd.DataFrame(student_data)

# Writing the DataFrame into an Excel file

df.to_excel("students.xlsx", sheet_name="Student Details", index=False)

print("Data successfully written to students.xlsx")
```

## Program 9: Reading an Excel File

```
import pandas as pd

# Reading an Excel file

df = pd.read_excel(

    "students.xlsx",

    sheet_name="Student Details"

)

print("Data read from Excel file:")

print(df)
```

## Sample Output

```
   Roll_No      Name Department  Percentage
0      101    Anusha         IT          89
1      102   Babitha         IT          92
2      103  Charitha        CSE          88
3      104   Deepika        ECE          85
```

## Program 10: Writing Multiple DataFrames into Different Excel Sheets

```
import pandas as pd

student_data = {
    "Roll_No": [101, 102, 103],
    "Name": ["Anusha", "Babitha", "Charitha"],
    "Department": ["IT", "IT", "CSE"],
    "Percentage": [89, 92, 88]
}
course_data = {
    "Course_ID": ["C101", "C102", "C103"],
    "Course_Name": ["Python", "Data Science", "Machine Learning"],
    "Credits": [4, 3, 4]
}
students_df = pd.DataFrame(student_data)
courses_df = pd.DataFrame(course_data)
# Writing multiple DataFrames to different sheets
with pd.ExcelWriter("college_data.xlsx", engine="openpyxl") as writer:
    students_df.to_excel(
        writer,
        sheet_name="Students",
```

```
        index=False
    )
    courses_df.to_excel(
        writer,
        sheet_name="Courses",
        index=False
    )
print("Multiple sheets successfully written to college_data.xlsx")
```

## Program 11: Reading All Sheets from an Excel File

```
import pandas as pd
# Reading all sheets
excel_data = pd.read_excel(
    "college_data.xlsx",
    sheet_name=None
)
print("Available sheets:")
print(excel_data.keys())
print("\nStudent Sheet:")
print(excel_data["Students"])
print("\nCourse Sheet:")
print(excel_data["Courses"])
```

# Complete Combined Program

```
import pandas as pd
from io import StringIO
# ---------------------------------------
```

```
# 1. CSV Reading and Writing

# ---------------------------------------

csv_url = "https://raw.githubusercontent.com/mwaskom/seaborn-data/master/iris.csv"

csv_df = pd.read_csv(csv_url)

print("CSV Data:")

print(csv_df.head())

csv_df.to_csv("iris_output.csv", index=False)

print("\nCSV file written successfully.")

# ---------------------------------------

# 2. Tab-Delimited File

# ---------------------------------------

student_data = {

    "Roll_No": [101, 102, 103, 104],

    "Name": ["Anusha", "Babitha", "Charitha", "Deepika"],

    "Department": ["IT", "IT", "CSE", "ECE"],

    "Percentage": [89, 92, 88, 85]

}

student_df = pd.DataFrame(student_data)

student_df.to_csv(

    "student_output.txt",

    sep="\t",

    index=False

)

tab_df = pd.read_csv(

    "student_output.txt",

    sep="\t"
```

```
)

print("\nTab-Delimited Data:")

print(tab_df)

# ----------------------------------------

# 3. JSON Reading and Writing

# ----------------------------------------

student_df.to_json(

    "student_output.json",

    orient="records",

    indent=4

)

json_df = pd.read_json("student_output.json")

print("\nJSON Data:")

print(json_df)

# Parsing JSON string

json_string = """

[

    {"Roll_No": 201, "Name": "Esha", "Percentage": 91},

    {"Roll_No": 202, "Name": "Farah", "Percentage": 87}

]

"""

parsed_json_df = pd.read_json(

    StringIO(json_string)

)

print("\nParsed JSON String:")

print(parsed_json_df)
```

```
# --------------------------------------
# 4. Excel Reading and Writing
# --------------------------------------
student_df.to_excel(
    "students.xlsx",
    sheet_name="Student Details",
    index=False
)
excel_df = pd.read_excel(
    "students.xlsx",
    sheet_name="Student Details"
)
print("\nExcel Data:")
print(excel_df)
print("\nAll file operations completed successfully.")
```

# Important Pandas Functions

| Function | Purpose |
|---|---|
| pd.read_csv() | Reads CSV or delimited text files |
| pd.read_table() | Reads tab-delimited files |
| DataFrame.to_csv() | Writes data to CSV or text files |
| pd.read_json() | Reads and parses JSON data |
| DataFrame.to_json() | Writes data to a JSON file |
| pd.read_excel() | Reads Microsoft Excel files |
| DataFrame.to_excel() | Writes data to Microsoft Excel files |

| | |
|---|---|
| pd.ExcelWriter() | Writes multiple DataFrames into multiple Excel sheets |

# Result

Thus, reading, writing, and parsing operations were successfully performed on CSV, tab-delimited text, JSON, and Microsoft Excel files using the Pandas library.